

# DevPilot AI

## Implementation Plan, Architecture, and Step-by-Step Execution Guide

**Project one-liner:** DevPilot AI helps engineering leads convert software requirements into architecture, implementation plans, code intelligence, review findings, security checks, testing strategy, and delivery-ready pull requests through a stateful multi-agent workflow with human approval for all write actions.

This guide is designed to turn the Week 3 agentic AI assignment into a portfolio-grade engineering project. It emphasizes stateful orchestration, tool use, failure recovery, and human-in-the-loop governance instead of a single LLM response or simple RAG lookup.

### 1. Executive Summary

Build DevPilot AI as an AI Engineering Lead and Delivery Command Center. A user submits a requirement or pull request; the system understands the request, analyzes the repository, proposes the architecture, creates an implementation plan, runs review/security/test agents, and prepares output for human approval.

The MVP should be small but complete: repository ingestion, LangGraph orchestration, agent state, RAG-backed tools, approval gates, error handling, and a working UI that shows the run trace.

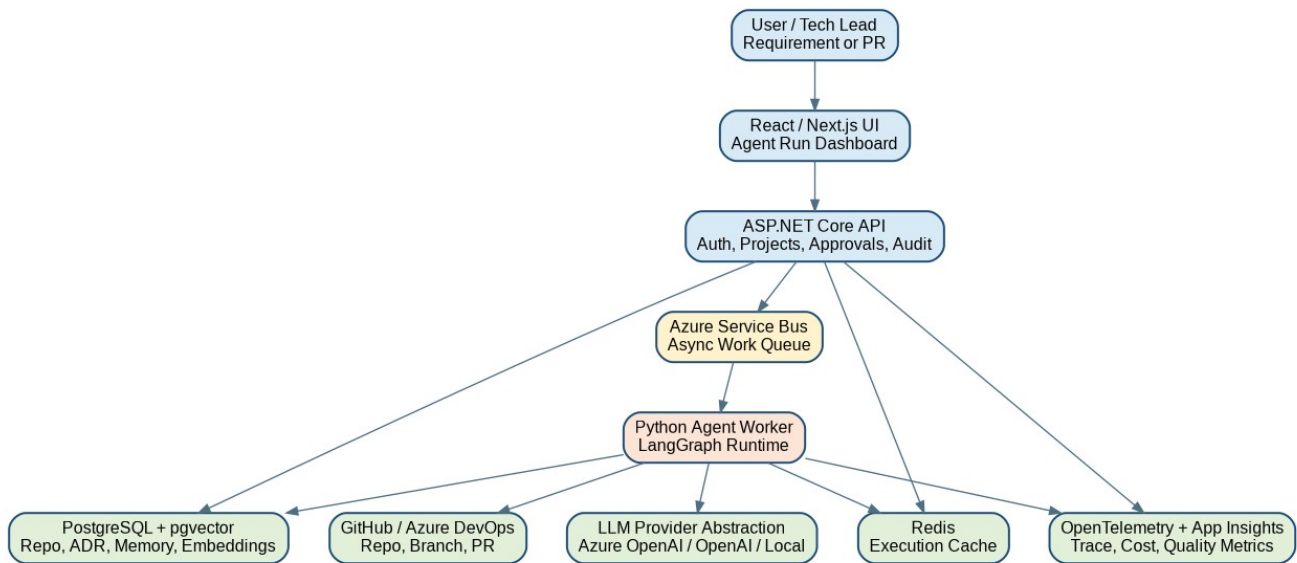
The portfolio version extends the MVP with ASP.NET Core APIs, Azure deployment, GitHub/Azure DevOps integration, observability, evaluation metrics, and architecture drift detection.

### 2. Product Goals and Non-Goals

| Goals                                        | Non-Goals                                                      |
|----------------------------------------------|----------------------------------------------------------------|
| Explainable multi-agent engineering workflow | No autonomous merge, deployment, payment, or destructive write |
| Repository-aware architecture and planning   | Not a generic chatbot or Copilot clone                         |
| Human approval before write actions          | No hidden agent actions without audit trail                    |
| Observable execution with traces and metrics | No black-box demo without state, logs, or evals                |
| Public portfolio quality documentation       | Do not overbuild every integration in the MVP                  |

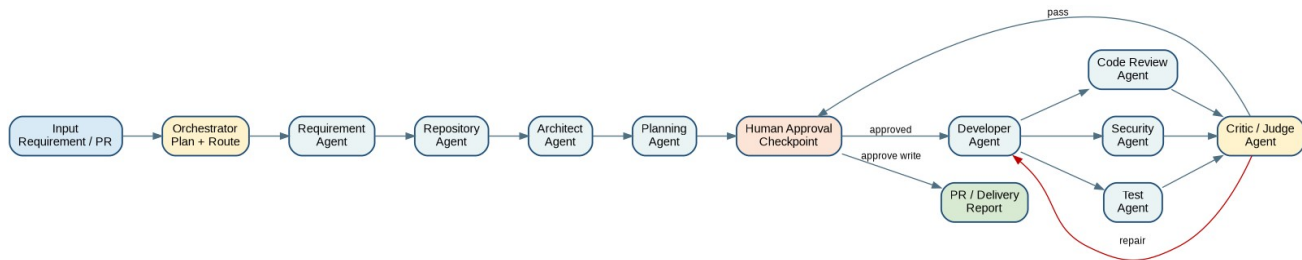
### 3. Target Architecture

The system is split into a .NET product API, a Python agent runtime, persistent memory, RAG/knowledge storage, Git integration, and observability. This separation lets you demonstrate both enterprise .NET architecture and modern AI orchestration.



## 4. Agentic Workflow

The orchestrator decides which specialized agent should act next. The flow must be resumable, inspectable, and recoverable. Write actions are gated by approval.

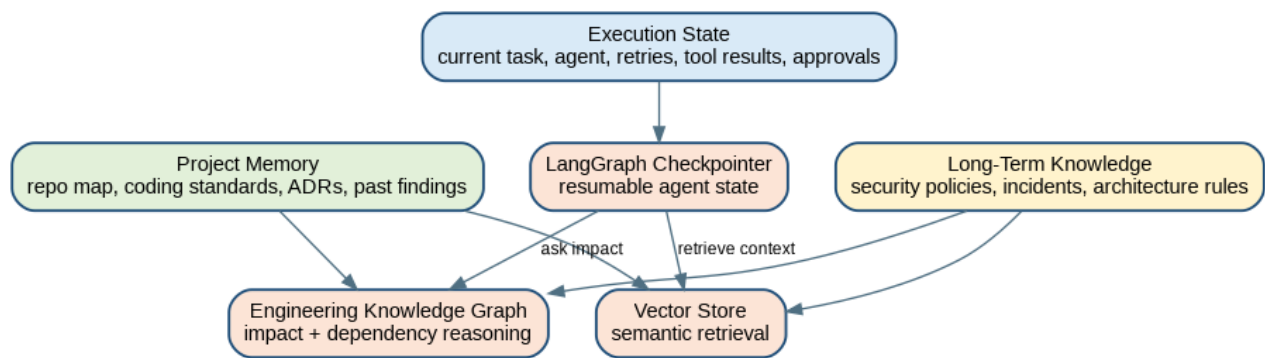


## Agent contracts

| Agent              | Responsibility                                                              | Permission boundary                            |
|--------------------|-----------------------------------------------------------------------------|------------------------------------------------|
| Orchestrator Agent | Owns graph state, routing, retries, and stopping conditions.                | Reads and routes; no direct writes.            |
| Requirement Agent  | Extracts objective, constraints, assumptions, risks, and missing questions. | Read-only.                                     |
| Repository Agent   | Maps projects, files, dependencies, and affected modules.                   | Read-only Git and code search.                 |
| Architect Agent    | Produces target design, architecture impact, and trade-offs.                | Read-only; may request approval for ADR draft. |
| Planning Agent     | Creates step-by-step implementation plan and acceptance criteria.           | Read-only until approved.                      |
| Developer Agent    | Generates candidate changes after plan approval.                            | Write only to working branch after approval.   |
| Code Review Agent  | Finds bugs, anti-patterns,                                                  | Read-only analysis.                            |

|                |                                                                      |                                       |
|----------------|----------------------------------------------------------------------|---------------------------------------|
|                | maintainability issues, and severity.                                |                                       |
| Security Agent | Checks secrets, auth flaws, injection risk, sensitive data handling. | Read-only analysis.                   |
| Test Agent     | Identifies missing unit/integration tests and proposes test cases.   | Read-only or approved write.          |
| Judge Agent    | Scores result against requirement, architecture rules, and tests.    | Read-only; routes repair or approval. |

### 5. State, Memory, and Knowledge



**Execution state:** Current requirement, selected agent, completed steps, tool outputs, retry count, error details, pending approvals, and final decision.

**Project memory:** Repository map, coding conventions, architecture decisions, known risks, dependency graph, and previous agent findings.

**Long-term knowledge:** Organization policies, secure coding standards, architecture rules, incidents, reusable patterns, and accepted decisions.

### 6. Implementation Roadmap

| Phase                             | Outcome                                                                                                                             |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Phase 0 - Setup                   | Create mono-repo, Docker Compose, base .NET API, Python agent service, database, and local configuration.                           |
| Phase 1 - Repository Intelligence | Clone or import a sample .NET repository, parse file tree, chunk code/docs, generate embeddings, and expose repository_search tool. |
| Phase 2 - LangGraph Orchestration | Define AgentState, graph nodes, conditional edges, retries, checkpoints, and structured outputs.                                    |
| Phase 3 - Core Agents             | Build requirement, repository, architect, planning, review, security, test, and judge agents with strict contracts.                 |

|                                   |                                                                                                                        |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Phase 4 - Human Approval          | Add approval objects, UI review screen, interrupts before write actions, and audit log.                                |
| Phase 5 - UI Dashboard            | Show project selection, requirement input, agent status, trace timeline, findings, approval prompts, and final report. |
| Phase 6 - Git Integration         | Read repo metadata, create branch after approval, generate patch, optionally create pull request.                      |
| Phase 7 - Observability and Evals | Track agent trace, tool success rate, latency, token/cost estimates, quality score, and failure paths.                 |
| Phase 8 - Azure Deployment        | Containerize services, deploy API/UI/worker/database/queue/cache, configure secrets and monitoring.                    |
| Phase 9 - Portfolio Polish        | Add README, architecture diagrams, demo script, screenshots, sample runs, evaluation results, and roadmap.             |

## 7. Step-by-Step Build Guide

- 1 Create the repository:** Create /src/api-dotnet, /src/agent-python, /src/web, /infra, /docs, /samples, and /evals. Add README, license, Docker Compose, and environment templates.
- 2 Build the NET API shell:** Add projects for Api, Application, Domain, Infrastructure, and Contracts. Implement health checks, Swagger, authentication placeholder, project endpoints, approval endpoints, and run history endpoints.
- 3 Build the Python agent shell:** Create FastAPI or worker service. Add LangGraph, structured output models, tool interfaces, LLM provider abstraction, configuration, and logging.
- 4 Add storage:** Use PostgreSQL for projects, runs, approvals, documents, embeddings metadata, and audit logs. Add pgvector for semantic search and Redis for short-lived execution cache.
- 5 Implement repository ingestion:** Clone or load a sample repository. Extract file metadata, chunk source files, summarize modules, embed chunks, and store file-to-module relationships.
- 6 Implement tools:** Create repository\_search, file\_read, dependency\_map, architecture\_rules\_search, git\_status, create\_patch, create\_branch, create\_pr, approval\_request, and eval\_score tools. Mark each as read or write.
- 7 Implement AgentState:** Track requirement, plan, selected files, findings, approvals, error count, tool outputs, final report, and termination status. Persist with LangGraph checkpointer.
- 8 Implement graph nodes:** Create nodes for requirement analysis, repo discovery, architecture plan, implementation plan, approval interrupt, implementation, review, security, test, judge, repair, and final report.
- 9 Add approval gates:** Before branch creation, patch application, PR creation, work item update, or any external write, interrupt and require explicit user approval.
- 10 Add UI execution trace:** Stream node status and tool events via SignalR or server-sent events. Show each agent step, retrieved evidence, generated decisions, and retry/error path.
- 11 Add failure handling:** For tool timeout, retry with backoff; for empty search, reformulate query; for missing context, ask user; for unsafe write, stop and escalate.

- 12. **12 Add evaluation suite:** Create sample tasks such as caching, authentication, logging, and API validation. Score output for completeness, architecture compliance, test coverage, security, and cost/latency.
- 13. **13 Containerize locally:** Use Docker Compose for API, web, agent worker, PostgreSQL, Redis, and optional local LLM gateway. Verify one command starts the full system.
- 14. **14 Deploy to Azure:** Use Azure Container Apps or App Service, Azure Database for PostgreSQL, Azure Cache for Redis, Service Bus, Key Vault, and Application Insights.
- 15. **15 Record demo:** Show requirement input, agent trace, architecture plan, approval checkpoint, review/security/test findings, retry handling, and final delivery report in under five minutes.

8. Recommended Repository Structure

devpilot-ai/  
src/  
  api-dotnet/  
    DevPilot.Api/  
    DevPilot.Application/  
    DevPilot.Domain/  
    DevPilot.Infrastructure/  
    DevPilot.Contracts/  
  agent-python/  
    devpilot\_agent/  
      graphs/  
      agents/  
      tools/  
      memory/  
      evals/  
  web/  
    app/  
    components/  
infra/  
  docker/  
  azure/  
  github-actions/  
docs/  
  architecture/  
  decisions/  
  demo/  
samples/  
  dotnet-commerce-api/  
evals/  
  tasks/  
  expected/

9. Minimal Data Model

| Entity  | Purpose                                          |
|---------|--------------------------------------------------|
| Project | Project metadata, repository URL, owner, default |

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
|                  | branch, status                                                             |
| AgentRun         | Requirement, run status, current node, timestamps, final outcome           |
| AgentStep        | Node name, agent name, input summary, output summary, duration, tool calls |
| ToolCall         | Tool name, permission type, input hash, output summary, success/failure    |
| Approval         | Requested action, risk level, approver, decision, timestamp                |
| DocumentChunk    | Repository/docs chunk, embedding, file path, line range, content type      |
| ArchitectureRule | Layering rules, dependency constraints, allowed/blocked patterns           |
| Finding          | Review/security/test finding, severity, evidence, suggested fix            |

## 10. Local Execution Guide

- **Prerequisites:** .NET SDK, Python 3.11+, Node.js, Docker Desktop, PostgreSQL/pgvector through Docker, GitHub token, LLM API key.
- **Start infrastructure:** docker compose up -d postgres redis servicebus-emulator or the chosen local queue replacement.
- **Run API:** dotnet run --project src/api-dotnet/DevPilot.Api/DevPilot.Api.csproj
- **Run agent worker:** cd src/agent-python && python -m devpilot\_agent.worker
- **Run web app:** cd src/web && npm install && npm run dev
- **Ingest sample repo:** Use the UI or CLI to load samples/dotnet-commerce-api and build the code index.
- **Start demo task:** Submit: Add Redis distributed caching to the Product API with cache-aside strategy and safe fallback.
- **Approve plan:** Review the architecture and implementation plan. Approve only the plan; keep code writes gated.
- **Review output:** Inspect findings, scorecard, proposed patch, and final delivery report.
- **Create PR:** Approve branch and PR creation only after review/security/test agents pass.

## 11. Security, Governance, and Guardrails

- **Tool permission model:** Every tool is classified as read, proposed-write, or write. Read tools can run autonomously; write tools require approval.
- **Prompt injection defense:** Treat repository content and documentation as untrusted data. Do not allow retrieved content to override system policies or tool permissions.
- **Secret handling:** Use Key Vault or local secrets. Never log API keys, tokens, connection strings, or private repository contents.
- **Audit trail:** Persist every run, step, tool call, approval, and final decision.
- **Least privilege:** Use read-only Git token for analysis; separate scoped token for approved write/PR operations.

## 12. MVP Acceptance Criteria

- User can create a project and ingest a sample .NET repository.
- User can submit an engineering requirement and see the graph execution trace.
- Requirement, repository, architect, planning, review, security, test, and judge agents produce structured output.
- Agent state persists and can be resumed after interruption.
- Read tools run autonomously; write tools require approval.
- At least one intentional failure path is demonstrated and handled.
- Final report includes architecture summary, affected files, risks, test plan, security findings, and recommended next action.
- GitHub README explains architecture, setup, demo flow, evaluation results, and roadmap.

## 13. Five-Minute Demo Script

| Time        | What to show                                                                                                    |
|-------------|-----------------------------------------------------------------------------------------------------------------|
| 0:00 - 0:30 | Introduce DevPilot AI as an agentic engineering workflow system, not a chatbot.                                 |
| 0:30 - 1:15 | Show architecture diagram and explain .NET API, Python LangGraph worker, database, vector store, queue, and UI. |
| 1:15 - 2:00 | Submit the sample requirement and show the orchestrator deciding the workflow.                                  |
| 2:00 - 3:00 | Show repository discovery, architecture proposal, and implementation plan.                                      |
| 3:00 - 3:45 | Show human approval before any write action.                                                                    |
| 3:45 - 4:30 | Show review/security/test agents and judge scorecard.                                                           |
| 4:30 - 5:00 | Show final report, failure handling, evaluation metrics, and GitHub roadmap.                                    |

## 14. Portfolio Roadmap

- **Architecture drift detection:** Compare code changes against defined layer and dependency rules.
- **Engineering knowledge graph:** Connect requirements, files, modules, tests, ADRs, findings, incidents, and deployment assets.
- **Azure DevOps/Jira integration:** Create/read work items, link PRs, and generate delivery status reports.
- **Cost and quality analytics:** Track token spend, latency, retry rate, approval rate, and output quality over time.
- **Multi-repo support:** Analyze changes spanning frontend, backend, infrastructure, and shared libraries.

## 15. Alignment With Week 3 Assignment

DevPilot AI satisfies the Week 3 objective because it is a stateful agentic system with tool use, control flow, failure handling, and human handoff. It also extends the suggested Code Review Agent into a stronger end-to-end software delivery workflow.

Expected submission artifacts: project documentation, a five-minute or shorter video demo, and a GitHub repository containing the codebase and setup instructions.